

Loss-Conditional PINNs for Parametric PDE Families

Anonymous Authors¹

Abstract

Physics-informed neural networks (PINNs) approximate solutions of ODEs and PDEs by minimising a weighted combination of residual, boundary, initial, and data losses. Their performance, however, is often dominated by the choice of loss weights: a poor weighting may lead training to a degenerate solution in which one physical constraint is satisfied while another is ignored. Existing methods therefore seek to select or adapt a single good set of weights. We take a different view: instead of tuning one weight vector, we explore the entire weight space during training.

We introduce LC-PINN, a loss-conditional physics-informed neural network in which the loss weights are treated as input parameters, sampled from a simple prior at every optimisation step. This turns PINN training into learning a continuous family of solutions indexed by the loss-weight vector, while requiring no solver-generated paired data. In this sense, LC-PINN lies between classical PINNs and operator-learning methods: it remains fully physics-informed, but amortises training over a parametric family.

We provide a simple theoretical justification for the loss-conditional construction and study it empirically on parametric Helmholtz, Schrödinger, viscous Burgers, and Buckley–Leverett equations. Across these problems, a single LC-PINN matches or improves retrained PINN baselines, parameterises full loss-weight families.

ary, initial, and (optional) data losses. Although highly flexible, PINNs are often extremely sensitive to the choice of loss weights and physical parameters. Poorly balanced objectives may lead optimization toward solutions that satisfy one physical constraint while largely ignoring others. As a result, practical PINN workflows frequently rely on careful hyperparameter tuning, adaptive weighting schedules, or even retraining for every parameter configuration of interest.

This paper addresses this limitation through a different perspective. Instead of searching for a single optimal weighting configuration, we train a single conditional PINN across an entire family of loss weights or PDE parameters. The key idea is to treat the conditioning variable λ as part of the network input, transforming PINN training into learning a continuous family of solutions indexed by λ .

The resulting model, which we call LC-PINN, occupies an intermediate position between classical PINNs and operator-learning methods. Like neural operators, LC-PINN amortizes computation across a family of PDE instances. Unlike operator-learning approaches such as FNO (Li et al., 2021) or DeepONet (Lu et al., 2021), however, LC-PINN remains fully residual-based and does not require solver-generated paired datasets.

Conceptually, the proposed framework rethinks loss balancing in PINNs. Conventional adaptive-weight methods attempt to identify a single good scalarization of the multi-objective training loss. LC-PINN instead explores the geometry of the entire loss-weight space during training. In this sense, loss weighting becomes not merely a hyperparameter-tuning problem, but a parametric learning problem.

Contributions.

- We introduce LC-PINN, a loss-conditional physics-informed neural network that treats loss weights and PDE parameters as conditioning variables, enabling a single model to represent a continuous family of PDE solutions.
- We provide a theoretical analysis of the LC-PINN objective, including a λ -invariance result showing that global minimizers of the conditional objective recover residual minimizers across the parameter family.
- We derive a local optimization result demonstrating

1. Introduction

Physics-Informed Neural Networks (PINNs; Raissi et al., 2019) approximate solutions of differential equations by minimizing a weighted combination of PDE residual, bound-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the AI4Physics Workshop at the International Conference on Machine Learning (AI4Physics@ICML 2026). Do not distribute.

that loss-conditioning introduces additional descent directions in parameter space, improving the instantaneous loss-decay rate in the quadratic regime.

- We evaluate LC-PINN on parametric Helmholtz, Schrödinger, Burgers, and Buckley–Leverett equations, demonstrating improved coherence across parameter families while amortizing training over the entire conditioning space.

2. Related work

Adaptive loss weighting in PINNs. The fragility of the composite-loss objective in the original PINN formulation (Raissi et al., 2019) has motivated a line of methods that adapt the residual / boundary / data weights during training. Self-adaptive PINN (SA-PINN) of McClenny & Braga-Neto (2020) attaches a trainable per-collocation-point weight that is updated by ascent on the residual, yielding an effective focusing of the network on hard-to-fit regions; their “points-with-weights” formulation is essentially a soft hard-example-mining for the residual. ReLoBRaLo (Bischof & Kraus, 2021) re-balances loss-term weights from running ratios of the absolute loss values, smoothed by an exponential moving average. Causal-PINN (Wang et al., 2024) weights the residual by a time-causal mask so the network does not collapse onto late-time dynamics before fitting the initial-time region. These all produce *one* solution for *one* (adaptively-found) weighting and would need to be retrained at every new weighting the practitioner wishes to inspect.

Operator learning. A parallel line of work has built neural operators that map a function-valued input (boundary condition, forcing term, initial condition) to a function-valued solution. The Fourier Neural Operator (FNO) (Li et al., 2021) parameterises the operator in spectral space: linear layers on the lowest Fourier modes, plus a residual 1×1 convolution. DeepONet (Lu et al., 2021) factorises the operator into a branch network (encoding the input function at sensor points) and a trunk network (encoding the spatial coordinate), recombined by an inner product. Both methods amortise across a family of problems but require a precomputed dataset of (input, solution) pairs from a classical solver—they are supervised learners in function space. The PINN residual itself plays no role in their training. PI-DeepONet (Wang et al., 2021) re-introduces the residual but stays inside the operator-learning framing, treating the input function as the parameter rather than a scalar coefficient. Our LC-PINN sits between these two camps: it amortises like operator learning but uses the residual loss like a PINN.

Loss-conditional networks. Dosovitskiy & Djolonga (2020) introduced the loss-conditional networks idea in image generation: a single generator, conditioned on the loss weights of a multi-objective composite reconstruction loss,

produces images that interpolate the tradeoff surface (sharpness vs. perceptual fidelity vs. adversarial realism). The network is trained by sampling the weight vector from a fixed prior at every step and applying the freshly-sampled weights to the per-instance reconstruction loss. LC-PINN takes the same conditional-on-weights idea and applies it to the PDE-residual setting; the formal λ -invariance result (Section 3.3) is, to our knowledge, the first analysis of the optimum-set structure of this construction in the residual-loss case.

Hypernetworks and meta-learning for PINNs. A separate amortisation strategy uses hypernetworks (Ha et al., 2017) or MAML-style meta-learning (Finn et al., 2017) to produce per-instance PINN weights. PI-MAML (Liu et al., 2022) demonstrates the meta-learning angle on parametric PDEs. These methods amortise at the level of network weights rather than a network input. They typically require a training distribution of full PDE *tasks* and a per-task inner-loop adaptation step at test time; LC-PINN trains a single network with no inner loop and produces predictions for every λ in one forward pass.

Extrapolation in λ vs. in physical parameters. In the loss-weight mode the network’s role is closest to Dosovitskiy & Djolonga (2020); in the parametric-coefficient mode it is closest to FNO / DeepONet but parameterised over a low-dimensional scalar input rather than a function-valued one. The two modes share an architecture; the choice of λ is task-driven.

3. Method

3.1. Loss-conditional formulation

A standard PINN (Raissi et al., 2019) approximates the solution of a single PDE $Fu = 0$ with auxiliary boundary, initial, and (optionally) data constraints $\{Bu = g_B, Iu = g_I, Du = g_D\}$ by minimizing the weighted composite loss

$$\begin{aligned} \mathcal{L}^{\text{PINN}}(\theta; w) = & w_F \|Fu_\theta\|_{L^2}^2 + w_B \|Bu_\theta - g_B\|_{L^2}^2 \\ & + w_I \|Iu_\theta - g_I\|_{L^2}^2 + w_D \|Du_\theta - g_D\|_{L^2}^2, \end{aligned} \quad (1)$$

with respect to the network parameters θ . The weight vector $w = (w_F, w_B, w_I, w_D)$ determines the tradeoff between satisfying the PDE residual and enforcing the auxiliary constraints. In practice, these weights are typically selected by grid search or adaptive reweighting strategies such as SA-PINN (McClenny & Braga-Neto, 2020), ReLoBRaLo (Bischof & Kraus, 2021), or Causal-PINN (Wang et al., 2024).

LC-PINN replaces this fixed-weight optimization problem with a continuous family of problems indexed by a conditioning variable $\lambda \in \Lambda$, where $\Lambda \subseteq \mathbb{R}^{d_\lambda}$ denotes ei-

ther (i) the space of admissible loss weights themselves ($\lambda = w$, $d_\lambda = 4$ for Burgers and Buckley–Leverett), or (ii) a physical PDE parameter such as a wavenumber or mobility ratio ($\lambda = k$, $d_\lambda = 1$ for Helmholtz). The resulting loss-conditional PINN (LC-PINN) is a single network

$$u_\theta : \Omega \times \Lambda \rightarrow \mathbb{R}, \quad (x, \lambda) \mapsto u_\theta(x, \lambda), \quad (2)$$

trained to minimize the expected loss over the parameter family:

$$J(\theta) = \mathbb{E}_{\lambda \sim p_\lambda} [\mathcal{L}(\lambda; u_\theta(\cdot, \lambda))], \quad (3)$$

where p_λ is a fixed sampling distribution over Λ (uniform unless stated otherwise).

Architecturally, the modification is minimal: the network receives an additional input of dimension d_λ . Computationally, each optimization step requires only a standard forward/backward pass at a freshly sampled λ . Conceptually, however, the formulation changes the role of loss balancing in PINNs. Instead of identifying a single optimal weighting configuration, the network is trained across the entire loss-weight space, learning a continuous family of solutions indexed by λ . This extends the loss-conditional networks of [Dosovitskiy & Djolonga \(2020\)](#) from generative loss-mixture modeling to the residual-loss setting of physics-informed learning.

3.2. Inference: K -shot averaging across λ

At evaluation time we report a single rel- L^2 error per task. In the loss-weight regime there is no canonical “true” choice of λ , so following [Dosovitskiy & Djolonga \(2020\)](#) we average predictions over K independent samples drawn from p_λ :

$$\hat{u}(x) = \frac{1}{K} \sum_{i=1}^K u_\theta(x, \lambda^{(i)}), \quad \lambda^{(i)} \stackrel{\text{i.i.d.}}{\sim} p_\lambda. \quad (4)$$

This K -shot averaging exposes the variability of LC-PINN across the loss-weight family and highlights the amortization advantage of the approach. A conventional PINN must be retrained K times to generate predictions for K different weight configurations, whereas LC-PINN produces all K predictions from a single training run.

3.3. λ -invariance at the LC optimum

The central hypothesis behind LC-PINN is that a single conditional network $u_\theta(x, \lambda)$, trained with $\lambda \sim p_\lambda$, can simultaneously parameterize an entire family of PDE solutions $\{F_\lambda u = 0\}_{\lambda \in \Lambda}$. The following proposition formalizes this statement at a global optimum of the LC-PINN objective.

Setup. Let $\Lambda \subseteq \mathbb{R}^{d_\lambda}$ denote the parameter space, p_λ a probability measure on Λ with full support, and $\Omega \subseteq \mathbb{R}^{d_x}$

the spatial–temporal domain equipped with sampling measure p_Ω . Each $\lambda \in \Lambda$ defines a residual operator F_λ together with associated boundary, initial, and optional data-fit operators $\{B_\lambda, I_\lambda, D_\lambda\}$. For a candidate field $v : \Omega \rightarrow \mathbb{R}$, define the corresponding per-parameter loss functional

$$\begin{aligned} \mathcal{L}(\lambda; v) = & w_F \|F_\lambda v\|_{L^2(\Omega, p_\Omega)}^2 + w_B \|B_\lambda v\|_{L^2(\partial\Omega)}^2 \\ & + w_I \|I_\lambda v\|_{L^2(t=0)}^2 + w_D \|D_\lambda v\|_{L^2}^2, \end{aligned}$$

where (w_F, w_B, w_I, w_D) are strictly positive coefficients. The LC-PINN objective is the expected loss over the parameter family:

$$J(\theta) = \int_\Lambda \mathcal{L}(\lambda; u_\theta(\cdot, \lambda)) dp_\lambda(\lambda).$$

We make the following assumptions:

- (A1) For every $\lambda \in \Lambda$, the network class contains a residual minimizer $u_\lambda^* \in \arg \min_v \mathcal{L}(\lambda; v)$ such that $\mathcal{L}(\lambda; u_\lambda^*) = 0$.
- (A2) The map $\lambda \mapsto \mathcal{L}(\lambda; u_\theta(\cdot, \lambda))$ is dp_λ -integrable for every θ .
- (A3) The network $u_\theta(x, \lambda)$ is jointly continuous in (x, λ) .

Proposition 3.1 (Residual minimizers at the LC optimum). *Let $\theta^* \in \arg \min_\theta J(\theta)$. Under assumptions (A1)–(A3), the section $u_{\theta^*}(\cdot, \lambda)$ is a residual minimizer for p_λ -almost every $\lambda \in \Lambda$:*

$$\mathcal{L}(\lambda; u_{\theta^*}(\cdot, \lambda)) = 0, \quad p_\lambda\text{-a.e. } \lambda \in \Lambda.$$

In particular, on $\text{supp}(p_\lambda)$ the LC-PINN satisfies the PDE residual $F_\lambda u_{\theta^}(\cdot, \lambda) = 0$ in $L^2(\Omega, p_\Omega)$ together with the associated auxiliary constraints.*

Sketch proof. For every θ and λ , the quantity $\mathcal{L}(\lambda; u_\theta(\cdot, \lambda))$ is nonnegative since each term is a squared L^2 norm. By assumption (A1), the minimum per-parameter loss equals zero, implying that $\inf_\theta J(\theta) \geq 0$ and that this lower bound is attainable whenever the network class can jointly represent the family $\{u_\lambda^*\}_{\lambda \in \Lambda}$.

Under assumption (A3), the universal approximation theorem for tanh networks (??) guarantees that, on the compact domain $\Omega \times \Lambda$, there exists $\theta^* \in \Theta$ such that

$$\sup_{(x, \lambda) \in \Omega \times \Lambda} |u_{\theta^*}(x, \lambda) - u_\lambda^*(x)|$$

is arbitrarily small, and therefore $J(\theta^*)$ can be made arbitrarily close to zero. Consequently,

$$0 = J(\theta^*) = \int_\Lambda \mathcal{L}(\lambda; u_{\theta^*}(\cdot, \lambda)) dp_\lambda(\lambda).$$

Since the integrand is nonnegative, the standard vanishing-integral argument implies

$$\mathcal{L}(\lambda; u_{\theta^*}(\cdot, \lambda)) = 0$$

for p_λ -almost every $\lambda \in \Lambda$. $\square$

Remark 1 (role of the support). The proposition is a p_λ -almost-everywhere statement. Consequently, LC-PINN may behave arbitrarily on p_λ -null sets, including boundary regions of Λ that receive no probability mass. In our experiments $p_\lambda = \text{Uniform}(\Lambda)$ has full support, so the p_λ -a.e. statement coincides with Lebesgue-a.e. on Λ .

Remark 2 (sampling-law invariance). If two probability measures p_λ and $\tilde{p}_\lambda$ share the same support $\Lambda_0 \subseteq \Lambda$, then their sets of a.e.-residual minimizers on Λ_0 coincide. Thus, the asymptotic optimum set of the LC objective depends only on the support of the sampling distribution, not on its precise density. At finite training budget, however, the choice of p_λ affects how gradient signal is distributed across parameter space, which we observe empirically in Section 5.6.

Remark 3 (Monte Carlo estimation). In practice, each optimization step estimates $\nabla_\theta J(\theta)$ using n_λ i.i.d. samples drawn from p_λ . The resulting Monte Carlo estimator is unbiased and has variance $O(1/n_\lambda)$. Empirically, $n_\lambda = 4$ provides a good tradeoff between gradient variance (small n_λ) and optimization progress per unit compute (large n_λ), see Section 5.6.

3.4. Network and training details

All experiments use fully-connected tanh networks with hidden widths [64, 64, 64, 64], Xavier initialization, and a single real-valued output head. The conditioning mechanism depends on the problem regime.

In the *loss-weight regime* (Burgers and Buckley–Leverett), the conditioning variable λ is concatenated directly to the spatial–temporal coordinates at the input layer, following the loss-conditional formulation of [Dosovitskiy & Djolonga \(2020\)](#). In the *parametric-coefficient regime* (1D/2D/3D Helmholtz and Schrödinger equations), λ is instead mapped to FiLM ([Perez et al., 2018](#)) scale-and-shift parameters that modulate each hidden layer. Training is then followed by an L-BFGS refinement phase equipped with a revert-on-worse safeguard.

Empirically, the FiLM+L-BFGS combination substantially improves performance in the parametric regime, reducing the $\text{rel-}L^2$ error by approximately one order of magnitude compared with simple input concatenation. The opposite behaviour is observed in the loss-weight regime, where plain concatenation remains more stable (Section D).

Optimization uses Adam with learning rate $\eta = 10^{-3}$, cosine learning-rate annealing, and gradient-norm clipping at 1.0. At each optimization step, parameters $\lambda \sim p_\lambda$ are sampled independently and the corresponding per-instance residual loss is evaluated on the current batch of collocation points. Before entering the network, λ is normalized to the interval $[-1, 1]$.

All experiments are implemented in PyTorch 2.9 and executed on an Apple M4 Max GPU through the MPS backend. Full hyperparameter settings and collocation budgets for each PDE family are reported in Section B.

4. Local effect of loss-conditioning on PINN optimization

PINNs are typically trained by minimizing a weighted combination of multiple nonnegative objectives, including PDE residual, boundary, initial-condition, and data losses:

$$\mathcal{L}_\lambda(\theta) = \sum_{i=1}^m \lambda_i \mathcal{L}_i(\theta), \quad \lambda_i \geq 0.$$

In the ideal well-specified setting, these objectives are compatible: there exists a parameter vector θ^* such that

$$\mathcal{L}_1(\theta^*) = \dots = \mathcal{L}_m(\theta^*) = 0.$$

Consequently, loss weighting does not change the target solution itself, but rather modifies the local optimization geometry around the shared optimum.

To analyze this effect, we consider a local quadratic approximation in a neighbourhood of θ^* . Assume that each loss component admits the second-order expansion

$$\mathcal{L}_i(\theta) = \frac{1}{2}(\theta - \theta^*)^\top H_i(\theta - \theta^*), \quad H_i \succeq 0, \quad i = 1, \dots, m.$$

For a fixed loss-weight vector λ , the resulting weighted PINN objective becomes

$$\mathcal{L}_\lambda(\theta) = \frac{1}{2}(\theta - \theta^*)^\top H_\lambda(\theta - \theta^*),$$

where

$$H_\lambda = \sum_{i=1}^m \lambda_i H_i.$$

The matrix H_λ determines the local conditioning of the optimization problem. In particular, small eigenvalues of H_λ correspond to weakly controlled directions in parameter space and may substantially slow gradient-based optimization.

We now compare this classical parameterization with the loss-conditional one. Let θ denote the original PINN parameters and let η denote the additional trainable parameters

introduced by conditioning the network on the loss-weight vector λ . For fixed λ , the conditional network can be viewed locally as a standard PINN with effective parameters

$$\theta_{\text{eff}}(\lambda; \theta, \eta) = \theta + B(\lambda, \eta),$$

where B is bilinear in λ and η . Equivalently, there exist matrices $M_1, \dots, M_m$ such that

$$B(\lambda, \eta) = \sum_{j=1}^m \lambda_j M_j \eta.$$

The following theorem shows that, at the same effective parameter vector, the loss-conditional parameterization possesses an instantaneous loss-decay rate at least as large as that of the standard PINN.

Theorem 4.1 (Local loss-decay advantage of loss-conditioning). *Fix λ and consider the local quadratic objective*

$$\mathcal{L}_\lambda(\vartheta) = \frac{1}{2}(\vartheta - \theta^*)^\top H_\lambda(\vartheta - \theta^*), \quad H_\lambda \succeq 0.$$

Consider two parameterizations of the same effective parameter vector ϑ :

$$\vartheta = \theta_{\text{reg}}$$

for the standard PINN, and

$$\vartheta = \theta_{\text{eff}}(\lambda; \theta, \eta) = \theta + B(\lambda, \eta)$$

for the loss-conditional PINN.

Suppose that, at some time instant, both parameterizations represent the same effective point:

$$\theta_{\text{reg}} = \theta_{\text{eff}}(\lambda; \theta, \eta) = \vartheta.$$

Then, under continuous-time gradient descent, the instantaneous decay rate of the loss in the loss-conditional parameterization satisfies

$$-\frac{d}{dt} \mathcal{L}_\lambda(\theta_{\text{eff}}(\lambda; \theta, \eta)) = -\frac{d}{dt} \mathcal{L}_\lambda(\theta_{\text{reg}}) + \|D_\eta B(\lambda, \eta)^\top \nabla_\vartheta \mathcal{L}_\lambda(\vartheta)\|^2.$$

In particular,

$$-\frac{d}{dt} \mathcal{L}_\lambda(\theta_{\text{eff}}(\lambda; \theta, \eta)) \geq -\frac{d}{dt} \mathcal{L}_\lambda(\theta_{\text{reg}}).$$

If

$$B(\lambda, \eta) = \sum_{j=1}^m \lambda_j M_j \eta,$$

then

$$D_\eta B(\lambda, \eta) = \sum_{j=1}^m \lambda_j M_j,$$

and the additional decay term becomes

$$\left\| \left(\sum_{j=1}^m \lambda_j M_j \right)^\top H_\lambda(\vartheta - \theta^*) \right\|^2.$$

The theorem provides a local explanation for the empirical robustness of LC-PINN. Loss-conditioning does not alter the common optimum θ^* ; instead, it enriches the parameterization through additional conditional directions in parameter space. Whenever the regular and conditional models represent the same effective PINN, the conditional model retains the original descent directions while also acquiring additional optimization directions through the conditioning parameters.

Consequently, in the local quadratic regime, the instantaneous loss decrease of the conditional model exceeds that of the standard PINN by a nonnegative squared-gradient term. Intuitively, this improvement arises because the optimization is performed in a higher-dimensional parameter space with additional degrees of freedom, allowing the model to identify more effective descent directions during training.

5. Results

We evaluate LC-PINN in two amortisation regimes. The first is the parametric-coefficient regime, where λ is a physical PDE parameter and the comparison with operator-learning methods is most direct. The second is the loss-weight regime, where λ is a loss-weight vector and LC-PINN is compared with adaptive PINN balancers. For the parametric-coefficient experiments, we report $\text{rel-}L^2$ error averaged over a fixed parameter grid. For the loss-weight experiments, we report K -shot averaged $\text{rel-}L^2$ error. Unless stated otherwise, all results are mean $\pm$ standard deviation over four random seeds; per-parameter baselines use two seeds when their retraining cost is substantially higher. Figure 1 summarizes the main wall-time/accuracy tradeoff.

5.1. 1D Helmholtz: parametric-coefficient regime

Table 1 reports $\text{rel-}L^2$ error averaged over the five-point grid $k \in \{1.00, 3.25, 5.50, 7.75, 10.00\}$. LC-PINN receives the wavenumber k as a normalized input, whereas the baselines are retrained separately for each value of k .

With FiLM conditioning on k and an L-BFGS refinement phase, a single LC-PINN covers the full interval $k \in [1, 10]$ with average $\text{rel-}L^2 \approx 9 \times 10^{-4}$. This improves over SA-PINN by approximately $3.4\times$ and over ReLoBRaLo by approximately $4.1\times$, while requiring less total wall time than running all five per- k retrains.

Table 1. 1D Helmholtz, $\text{rel-}L^2$ averaged across $k \in \{1.00, 3.25, 5.50, 7.75, 10.00\}$. LC-PINN is one trained network evaluated at each k ; baselines are five separate retrainsings, one per k .

Method	$\text{rel-}L^2$ (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$9.39 \times 10^{-4} \pm 1.18 \times 10^{-4}$	65.9 min/seed
PI-DeepONet (one network)	$2.21 \times 10^{-2} \pm 3.18 \times 10^{-2}$	61.8 min/seed
SA-PINN (per- k , 5 retrainsings)	$3.23 \times 10^{-3} \pm 2.90 \times 10^{-3}$	11.4 min/ k
ReLoBraLo (per- k , 5 retrainsings)	$3.87 \times 10^{-3} \pm 2.82 \times 10^{-3}$	7.9 min/ k

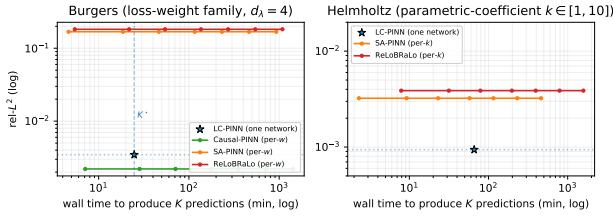


Figure 1. Wall-time/accuracy frontier as the number K of evaluated λ values increases. LC-PINN trains once and produces predictions for any K , while per- λ baselines move rightward by a factor of K . On Helmholtz, LC-PINN Pareto-dominates from $K = 1$. On Burgers, Causal-PINN is more accurate for a single configuration, but becomes more expensive than LC-PINN once $K \gtrsim 4$.

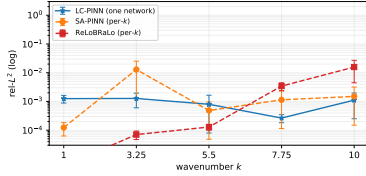


Figure 2. Per- k $\text{rel-}L^2$ on 1D parametric Helmholtz. Error bars show $\pm$ standard deviation over four seeds. Retrained baselines vary strongly across the parameter grid, whereas LC-PINN remains comparatively stable.

Per-parameter breakdown. The grid average hides an important difference between the methods. Table 2 reports $\text{rel-}L^2$ error at each value of k . Retrained baselines can be extremely accurate at easy parameter values: for example, ReLoBraLo reaches 5.75×10^{-6} at $k = 1$. However, their performance varies sharply across the family. At $k = 10$, ReLoBraLo deteriorates to 1.57×10^{-2} , three orders of magnitude worse than its own $k = 1$ result. SA-PINN exhibits similar instability, with its worst result occurring at $k = 3.25$.

By contrast, LC-PINN remains within one order of magnitude of its best performance across the entire grid. This highlights the main empirical tradeoff: amortisation may sacrifice peak accuracy at the easiest single parameter values, but it provides substantially greater coherence across the full parameter family. LC-PINN is therefore aimed at parametric settings where solutions are needed for many values of k , rather than at single-parameter optimization.

Overall, the Helmholtz experiment shows that LC-PINN improves the metric that matters in the parametric setting: grid-averaged and worst-case accuracy across the family.

5.2. 1D Schrödinger: parametric-coefficient regime

We next test whether the improvement in Section 5.1 is specific to the Helmholtz operator. We consider a 1D stationary Schrödinger problem with parametric harmonic potential,

$$-u'' + \alpha^2(x - \frac{1}{2})^2 u = f(x; \alpha), \quad x \in (0, 1),$$

with homogeneous Dirichlet boundary conditions and $\alpha \in [0.5, 10]$. Compared with Helmholtz, the parameter α multiplies a spatially varying potential rather than a constant zeroth-order term. The manufactured solution is $\sin(\pi x) \exp(-\alpha(x - \frac{1}{2})^2/2)$; details of the forcing are given in Section B.

LC-PINN improves over the per- α SA-PINN baseline by roughly $20\times$ on the grid mean. The per- α breakdown (Table 9) again shows that retrained baselines may perform well at individual parameter values but can be unstable across the family: at $\alpha = 5.0$, the two SA-PINN seeds differ by two orders of magnitude. LC-PINN maintains seed-to-seed standard deviation below 3.2×10^{-4} throughout $\alpha \in [0.5, 10]$.

PI-DeepONet matches LC-PINN on this smoother parametric potential, whereas LC-PINN substantially outperforms PI-DeepONet on Helmholtz. This suggests that amortisation is robust across architectures for smooth potentials, while high-frequency wave operators particularly benefit from FiLM-based conditioning.

5.3. 2D Helmholtz: parametric-coefficient regime

We extend the Helmholtz experiment to the unit square:

$$\Delta u + k^2 u = f(x, y; k), \quad (x, y) \in [0, 1]^2,$$

with homogeneous Dirichlet boundary conditions and manufactured solution

$$u(x, y; k) = \sin(\pi x) \sin(\pi y) \cos(kx) \cos(ky).$$

The same LC-PINN architecture conditions on the normalized wavenumber k , while the baselines are retrained separately for each training value of k .

Table 2. Per- k rel- L^2 on 1D parametric Helmholtz (mean $\pm$ std over four seeds). Bold indicates the best method per row. LC-PINN is one trained network evaluated at each k ; baselines are retrained separately for each k .

k	SA-PINN (per- k)	ReLoBRaLo (per- k)	LC-PINN (one net)
1.00	$(1.23 \pm 0.61) \times 10^{-4}$	$(5.75 \pm 1.73) \times 10^{-6}$	$(1.25 \pm 0.37) \times 10^{-3}$
3.25	$(1.29 \pm 1.23) \times 10^{-2}$	$(7.04 \pm 2.31) \times 10^{-5}$	$(1.27 \pm 0.67) \times 10^{-3}$
5.50	$(4.83 \pm 4.75) \times 10^{-4}$	$(1.29 \pm 0.30) \times 10^{-4}$	$(7.97 \pm 8.53) \times 10^{-4}$
7.75	$(1.14 \pm 1.16) \times 10^{-3}$	$(3.42 \pm 1.04) \times 10^{-3}$	$(2.64 \pm 0.82) \times 10^{-4}$
10.00	$(1.50 \pm 1.68) \times 10^{-3}$	$(1.57 \pm 1.13) \times 10^{-2}$	$(1.11 \pm 0.86) \times 10^{-3}$
grid mean	3.23×10^{-3}	3.87×10^{-3}	9.39×10^{-4}

Table 3. 1D Schrödinger equation with parametric harmonic well, rel- L^2 averaged across $\alpha \in \{0.5, 5.0, 10.0\}$. LC-PINN is one trained network evaluated at each α ; SA-PINN is retrained separately for each α .

Method	rel- L^2 (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$1.99 \times 10^{-4} \pm 1.02 \times 10^{-4}$	46.6 min/seed
PI-DeepONet (one network)	$1.79 \times 10^{-4} \pm 3.70 \times 10^{-5}$	40.3 min/seed
SA-PINN (per- α , 3 retrainings)	$3.89 \times 10^{-3} \pm 3.70 \times 10^{-3}$	0.8 min/ α

Table 4. 2D Helmholtz on the unit square, rel- L^2 averaged across $k \in \{1.00, 5.50, 10.00\}$. LC-PINN is one trained network evaluated at each k ; baselines are retrained separately for each k .

Method	rel- L^2 (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$2.36 \times 10^{-2} \pm 2.56 \times 10^{-2}$	85.9 min/seed
SA-PINN (per- k , 3 retrainings)	$7.83 \times 10^{-2} \pm 2.76 \times 10^{-2}$	3.0 min/ k
ReLoBRaLo (per- k , 3 retrainings)	$1.13 \times 10^{-1} \pm 1.72 \times 10^{-2}$	5.6 min/ k

The 2D problem has a richer solution structure, with boundary nodal lines and tensor-product oscillations in both spatial directions. Nevertheless, the same LC-PINN construction transfers without modification. It reaches average rel- $L^2 \approx 2.4 \times 10^{-2}$, outperforming SA-PINN by approximately $3.3\times$ and ReLoBRaLo by approximately $4.8\times$.

5.4. 3D Helmholtz: parametric-coefficient regime

Finally, we consider 3D Helmholtz on the unit cube,

$$\Delta u + k^2 u = f(x, y, z; k), \quad (x, y, z) \in [0, 1]^3,$$

with homogeneous Dirichlet boundary conditions and tensor-product manufactured solution

$$u(x, y, z; k) = \prod_{q \in \{x, y, z\}} \sin(\pi q) \cos(kq).$$

The evaluation grid is $k \in \{1, 3, 5\}$; the high- k regime becomes substantially harder in 3D because oscillatory complexity increases in all spatial directions.

LC-PINN again remains the strongest method on the grid mean, improving over SA-PINN by approximately $4.2\times$ and over ReLoBRaLo by approximately $6.2\times$. This confirms that the parametric conditioning approach extends from 1D to 2D and 3D without changing the core method.

5.5. Loss-weight regime: Burgers and Buckley–Leverett

We now turn to loss-weight conditioning, where $\lambda \in \Delta^3$ parameterizes the relative weights of the PINN loss components. In this regime, FiLM modulation and L-BFGS refinement do not improve stability; we therefore use the simpler concat+Adam configuration following [Dosovitskiy & Dzhong \(2020\)](#). Table 6 reports rel- L^2 error on viscous Burgers with $\nu = 0.01/\pi$, initial condition $u(0, x) = -\sin(\pi x)$, and final time $t = 1$.

LC-PINN matches the strongest single-shot baseline, Causal-PINN, within a factor of two and outperforms SA-PINN and ReLoBRaLo by more than an order of magnitude. Its main advantage is amortisation: once trained, the same network produces predictions for arbitrary loss weights. The amortisation factor $A(K) = K T_{\text{base}}/T_{\text{LC}}$ breaks even at $K^* \approx 3.5$ – 5.4 and reaches roughly 19 – $29\times$ at $K = 100$.

On viscous Buckley–Leverett with $\varepsilon = 10^{-2}$ (Table 7), ReLoBRaLo is approximately $2\times$ more accurate than LC-PINN at a single weighting, while SA-PINN fails to converge below rel- $L^2 \approx 0.5$. In the zero-viscosity formulation, all PINN-family methods saturate near this error level.

Table 5. 3D Helmholtz on the unit cube, $\text{rel-}L^2$ averaged across $k \in \{1, 3, 5\}$. LC-PINN is one trained network evaluated at each k ; baselines are retrained separately for each k .

Method	$\text{rel-}L^2$ (mean $\pm$ std)	wall time
LC-PINN (FiLM+L-BFGS, one network)	$1.67 \times 10^{-2} \pm 1.97 \times 10^{-3}$	228.9 min/seed
SA-PINN (per- k , 3 retrainings)	$6.97 \times 10^{-2} \pm 2.47 \times 10^{-3}$	5.7 min/ k
ReLoBRaLo (per- k , 3 retrainings)	$1.03 \times 10^{-1} \pm 7.82 \times 10^{-3}$	11.9 min/ k

Table 6. Burgers, $\text{rel-}L^2$ at the test initial condition $-\sin(\pi x)$. LC-PINN reports $K = 100$ -shot averaging across loss weights; baselines are single-shot at their learned or self-adapted weighting.

Method	$\text{rel-}L^2$ (mean $\pm$ std)	wall time / seed
LC-PINN ($K = 100$)	$3.47 \times 10^{-3} \pm 2.73 \times 10^{-3}$	24.8 min
Causal-PINN	$2.21 \times 10^{-3} \pm 6.72 \times 10^{-4}$	7.1 min
SA-PINN	$1.68 \times 10^{-1} \pm 3.87 \times 10^{-2}$	4.6 min
ReLoBRaLo	$1.82 \times 10^{-1} \pm 1.71 \times 10^{-2}$	5.4 min
FNO (operator)	$5.37 \times 10^{-1} \pm 1.89 \times 10^{-1}$	8.0 min

Table 7. Viscous-regularized Buckley–Leverett ($\varepsilon = 10^{-2}$), $\text{rel-}L^2$ at the test snapshots. The reference is produced by an in-house finite-volume solver and used only for evaluation. LC-PINN reports $K = 25$ -shot averaging over loss weights; baselines are single-shot.

Method	$\text{rel-}L^2$ (mean $\pm$ std)
LC-PINN ($K = 25$)	$1.03 \times 10^{-2} \pm 1.30 \times 10^{-3}$
SA-PINN	$4.60 \times 10^{-1} \pm 6.74 \times 10^{-2}$
ReLoBRaLo	$5.09 \times 10^{-3} \pm 7.79 \times 10^{-4}$

5.6. Ablations

We perform three ablations on Burgers; full results are reported in Section C. First, increasing the number of evaluation samples K has little effect: K -shot averaging remains stable across $K \in \{25, \dots, 400\}$ at approximately 3.5×10^{-3} . Second, the number of parameter samples per optimization step is non-monotone, with the best performance at $n_\lambda = 4$. Third, replacing uniform sampling of λ with a log-uniform distribution degrades performance by approximately 14 $\times$, highlighting the importance of finite-budget coverage of the loss-weight space.

6. Discussion

We introduced LC-PINN, a loss-conditional physics-informed neural network that treats loss weights and PDE parameters as conditioning variables rather than fixed hyperparameters. Instead of searching for a single optimal weighting configuration, LC-PINN learns a continuous family of solutions indexed by λ . This rethinks loss balancing in PINNs: rather than tuning one scalarization of a multi-objective objective, the network is trained across the entire loss-weight space.

Empirically, LC-PINN produces more coherent be-

haviour across parameter families than conventional per-configuration retraining. In the parametric-coefficient regime, FiLM-conditioned LC-PINNs achieve stable accuracy across Helmholtz and Schrödinger families while avoiding the strong variability observed in per- k retrained baselines. In the loss-weight regime, a single LC-PINN approaches the performance of adaptive-weight PINNs while amortizing training across the full family of loss weights. The local analysis in Section 4 suggests that conditioning improves optimization geometry by introducing additional descent directions in parameter space.

Conceptually, LC-PINN occupies an intermediate position between classical PINNs and operator-learning methods. Like neural operators, it amortizes computation across PDE families, but unlike operator-learning approaches it remains fully residual-based and does not require solver-generated paired datasets. The current formulation nevertheless becomes more difficult in regimes with shocks, bifurcations, or highly oscillatory solutions, where jointly approximating the full parameter family is challenging. Future work includes adaptive sampling strategies and extensions to inverse and multi-physics problems.

Overall, the results suggest that conditioning on loss geometry and PDE parameters can transform PINNs from single-instance solvers into amortized parametric surrogate models.

References

- Bischof, R. and Kraus, M. A. Multi-objective loss balancing for physics-informed deep learning. *arXiv preprint arXiv:2110.09813*, 2021.
- Dosovitskiy, A. and Djourou, J. You only train once: loss-conditional training of deep networks. In *International*

- Conference on Learning Representations (ICLR)*, 2020.
- Finn, C., Abbeel, P., and Levine, S. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*, 2017.
- Ha, D., Dai, A., and Le, Q. V. Hypernetworks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
- Liu, X., Zhang, X., Peng, W., Zhou, W., and Yao, W. A novel meta-learning initialization method for physics-informed neural networks. *Neural Computing and Applications*, 34:14511–14534, 2022.
- Lu, L., Jin, P., Pang, G., Zhang, Z., and Karniadakis, G. E. Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021.
- McClenny, L. and Braga-Neto, U. Self-adaptive physics-informed neural networks using a soft attention mechanism. *arXiv preprint arXiv:2009.04544*, 2020.
- Perez, E., Strub, F., de Vries, H., Dumoulin, V., and Courville, A. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2018.
- Raissi, M., Perdikaris, P., and Karniadakis, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- Wang, S., Wang, H., and Perdikaris, P. Learning the solution operator of parametric partial differential equations with physics-informed deepnets. *Science Advances*, 7(40): eabi8605, 2021.
- Wang, S., Sankaran, S., and Perdikaris, P. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024.

A. Derivation of the local loss-decay result

This appendix provides the details behind Theorem 4.1. The goal is not to give a global optimization guarantee, but to formalize a local mechanism by which loss-conditioning can improve the instantaneous decay of a weighted PINN loss near a common optimum.

A.1. Local quadratic model for multi-objective PINN losses

A PINN objective is typically a weighted combination of several nonnegative loss components:

$$\mathcal{L}_\lambda(\theta) = \sum_{i=1}^m \lambda_i \mathcal{L}_i(\theta), \quad \lambda_i \geq 0.$$

The components may correspond to the PDE residual, boundary conditions, initial conditions, and data mismatch. In the well-specified case, there exists a common solution θ^* such that

$$\mathcal{L}_i(\theta^*) = 0, \quad i = 1, \dots, m.$$

Therefore, all positive scalarizations share the same global zero-loss solution.

Near θ^* , we approximate each component by its second-order Taylor expansion. Since θ^* is a local minimizer of each nonnegative loss component, the first-order term vanishes. Thus, locally,

$$\mathcal{L}_i(\theta) = \frac{1}{2}(\theta - \theta^*)^\top H_i(\theta - \theta^*), \quad H_i \succeq 0.$$

The weighted loss is then

$$\mathcal{L}_\lambda(\theta) = \sum_{i=1}^m \lambda_i \mathcal{L}_i(\theta) = \frac{1}{2}(\theta - \theta^*)^\top H_\lambda(\theta - \theta^*),$$

where

$$H_\lambda = \sum_{i=1}^m \lambda_i H_i.$$

Thus, choosing the loss weights is equivalent, locally, to choosing the effective Hessian H_λ seen by the optimizer.

For the regular PINN parameterization, the gradient flow is

$$\dot{\theta}(t) = -\nabla_\theta \mathcal{L}_\lambda(\theta(t)) = -H_\lambda(\theta(t) - \theta^*).$$

If $H_\lambda \succ 0$, then

$$\mathcal{L}_\lambda(\theta(t)) \leq e^{-2\lambda_{\min}(H_\lambda)t} \mathcal{L}_\lambda(\theta(0)).$$

Hence, the local convergence rate is controlled by the smallest eigenvalue of H_λ . If H_λ is nearly singular, convergence can be slow in weakly controlled directions.

A.2. Effective parameters induced by loss-conditioning

We now describe the local parameterization induced by conditioning the network on the loss-weight vector λ .

Let $\theta \in \mathbb{R}^p$ denote the parameters of the original PINN, and let $\eta \in \mathbb{R}^q$ denote the additional trainable parameters introduced by the conditional architecture. For example, if the first layer of a standard PINN is

$$z_1 = W_x x + b,$$

then concatenating λ to the input gives

$$z_1 = W_x x + W_\lambda \lambda + b.$$

For fixed λ , the term $W_\lambda \lambda$ acts as a λ -dependent shift of the first-layer bias:

$$b_{\text{eff}}(\lambda) = b + W_\lambda \lambda.$$

Thus, after freezing λ , the conditional network can be viewed locally as an ordinary PINN with λ -dependent effective parameters.

More generally, we model this effect by

$$\theta_{\text{eff}}(\lambda; \theta, \eta) = \theta + B(\lambda, \eta),$$

where B is bilinear in the conditioning variable λ and in the additional trainable parameters η . Equivalently, there exist matrices

$$M_1, \dots, M_m \in \mathbb{R}^{p \times q}$$

such that

$$B(\lambda, \eta) = \sum_{j=1}^m \lambda_j M_j \eta.$$

The dependence on λ is linear because the conditioning variables enter the network input linearly, while the dependence on η is linear because η represents the trainable weights connecting the conditioning variables to the network.

Under this local model, the loss-conditional objective for fixed λ becomes

$$\mathcal{L}_\lambda(\theta_{\text{eff}}(\lambda; \theta, \eta)) = \frac{1}{2}(\theta + B(\lambda, \eta) - \theta^*)^\top H_\lambda(\theta + B(\lambda, \eta) - \theta^*).$$

If the training samples λ from a distribution $p(\lambda)$, the corresponding expected local objective is

$$\mathcal{J}(\theta, \eta) = \mathbb{E}_{\lambda \sim p} \left[\frac{1}{2}(\theta + B(\lambda, \eta) - \theta^*)^\top H_\lambda(\theta + B(\lambda, \eta) - \theta^*) \right].$$

The theorem in the main text studies the instantaneous loss decay for a fixed value of λ .

A.3. Proof of Theorem 4.1

Fix λ and define the effective parameter

$$\vartheta = \theta_{\text{eff}}(\lambda; \theta, \eta) = \theta + B(\lambda, \eta).$$

The local quadratic loss is

$$\mathcal{L}_\lambda(\vartheta) = \frac{1}{2}(\vartheta - \theta^*)^\top H_\lambda(\vartheta - \theta^*).$$

Let

$$g = \nabla_\vartheta \mathcal{L}_\lambda(\vartheta).$$

Since the loss is quadratic,

$$g = H_\lambda(\vartheta - \theta^*).$$

Regular PINN. For the regular PINN, the effective parameter is simply

$$\vartheta = \theta_{\text{reg}}.$$

The gradient-flow dynamics is

$$\dot{\theta}_{\text{reg}} = -g.$$

Therefore,

$$\frac{d}{dt} \mathcal{L}_\lambda(\theta_{\text{reg}}) = g^\top \dot{\theta}_{\text{reg}} = -\|g\|^2.$$

Hence,

$$-\frac{d}{dt} \mathcal{L}_\lambda(\theta_{\text{reg}}) = \|g\|^2.$$

Loss-conditional PINN. For the loss-conditional PINN,

$$\vartheta = \theta + B(\lambda, \eta).$$

For fixed λ , the gradients with respect to the trainable variables θ and η are obtained by the chain rule:

$$\nabla_{\theta} \mathcal{L}_{\lambda} = g,$$

because $\partial \vartheta / \partial \theta$ is the identity, and

$$\nabla_{\eta} \mathcal{L}_{\lambda} = D_{\eta} B(\lambda, \eta)^{\top} g.$$

The gradient-flow dynamics in the extended parameter space is therefore

$$\dot{\theta} = -g, \quad \dot{\eta} = -D_{\eta} B(\lambda, \eta)^{\top} g.$$

The loss decay along this flow is

$$\frac{d}{dt} \mathcal{L}_{\lambda}(\theta + B(\lambda, \eta)) = -\|\nabla_{\theta} \mathcal{L}_{\lambda}\|^2 - \|\nabla_{\eta} \mathcal{L}_{\lambda}\|^2.$$

Substituting the expressions for the gradients gives

$$\frac{d}{dt} \mathcal{L}_{\lambda}(\theta + B(\lambda, \eta)) = -\|g\|^2 - \|D_{\eta} B(\lambda, \eta)^{\top} g\|^2.$$

Therefore,

$$-\frac{d}{dt} \mathcal{L}_{\lambda}(\theta + B(\lambda, \eta)) = \|g\|^2 + \|D_{\eta} B(\lambda, \eta)^{\top} g\|^2.$$

Since

$$-\frac{d}{dt} \mathcal{L}_{\lambda}(\theta_{\text{reg}}) = \|g\|^2,$$

we obtain

$$-\frac{d}{dt} \mathcal{L}_{\lambda}(\theta + B(\lambda, \eta)) = -\frac{d}{dt} \mathcal{L}_{\lambda}(\theta_{\text{reg}}) + \|D_{\eta} B(\lambda, \eta)^{\top} g\|^2.$$

This proves the claimed identity.

Finally, since

$$g = H_{\lambda}(\vartheta - \theta^*),$$

we have

$$\|D_{\eta} B(\lambda, \eta)^{\top} g\|^2 = \|D_{\eta} B(\lambda, \eta)^{\top} H_{\lambda}(\vartheta - \theta^*)\|^2.$$

If

$$B(\lambda, \eta) = \sum_{j=1}^m \lambda_j M_j \eta,$$

then

$$D_{\eta} B(\lambda, \eta) = \sum_{j=1}^m \lambda_j M_j.$$

Hence the additional decay term becomes

$$\left\| \left(\sum_{j=1}^m \lambda_j M_j \right)^{\top} H_{\lambda}(\vartheta - \theta^*) \right\|^2.$$

This completes the proof.

A.4. Interpretation

The result shows that, in the local quadratic regime, the conditional architecture has a larger instantaneous descent capacity than the regular PINN at the same effective parameter vector. The regular PINN can move only through the original parameter direction θ , whereas the loss-conditional PINN can additionally move through the conditional parameters η . These extra directions contribute the nonnegative term

$$\|D_\eta B(\lambda, \eta)^\top \nabla_\vartheta \mathcal{L}_\lambda(\vartheta)\|^2$$

to the instantaneous loss-decay rate.

This term vanishes only when the loss gradient is orthogonal to the subspace generated by the conditional parameters. Otherwise, the loss-conditional model decreases the same local quadratic loss strictly faster at the same effective point.

The statement should be interpreted locally. It does not imply that loss-conditioning changes the global optimum: in the well-specified case, the common target remains θ^* . Rather, the theorem shows that conditioning enriches the parameterization through which the optimizer approaches this optimum. This provides a possible explanation for the empirical observation that LC-PINN is less sensitive to a single poor loss-weight choice and can explore a family of loss-weight geometries during training.

A.5. Remark on discrete gradient descent

The theorem is stated for continuous-time gradient descent. For discrete gradient descent with step size α , the same comparison holds to first order in α . Indeed, for a smooth loss F , one step of gradient descent gives

$$F(z - \alpha \nabla F(z)) = F(z) - \alpha \|\nabla F(z)\|^2 + O(\alpha^2).$$

Applying this expansion to the regular and loss-conditional parameterizations yields the same leading-order difference:

$$\alpha \|D_\eta B(\lambda, \eta)^\top \nabla_\vartheta \mathcal{L}_\lambda(\vartheta)\|^2 + O(\alpha^2).$$

Thus, for sufficiently small step sizes, the loss-conditional parameterization has a larger one-step decrease whenever the additional gradient component is nonzero.

B. Hyperparameters and training details

This appendix collects the hyperparameters used for every method in ???. All numbers are the configuration that generated the headline tables in Section 5.

Shared LC-PINN configuration. Network: 4-layer MLP, hidden widths [64, 64, 64, 64], tanh activations, Xavier initialisation. Conditioning route is mode-dependent: concat at the input layer for the loss-weight mode (Burgers, BL); FiLM (Perez et al., 2018) on every hidden layer followed by an L-BFGS finishing pass with a revert-on-worse safety check for the parametric-coefficient mode (Helmholtz, Schrödinger). Optimiser: Adam, $\eta = 10^{-3}$, cosine-annealed to 10^{-5} over the full budget, gradient clip 1.0. Mini-batch composition: $n_\lambda = 4$ parameter samples per step, each with the per-equation collocation counts below. Backend: PyTorch 2.9 with the MPS device on a single Apple M4 Max.

Per-equation collocation and budgets. Table 8 lists the LC-PINN collocation counts and step budgets per PDE family.

Baseline configurations. All single- λ baselines share the LC-PINN backbone (same widths, activation, initialisation, and Adam schedule) so that the amortisation comparison is on equal architectural footing.

- *SA-PINN* (McClenny & Braga-Neto, 2020): per-collocation-point trainable weight ascended on the residual; Adam phase 10k iterations, then weights frozen and L-BFGS phase 5k iterations on the unweighted loss.
- *ReLoBRaLo* (Bischof & Kraus, 2021): 3-term loss-weight balancer (residual / boundary / initial) with softmax over log-loss-ratios, smoothed by EMA. Hyperparameters $\alpha = 0.999$, $\tau = 0.1$, $\rho_{\text{mean}} = 0.999$.
- *Causal-PINN* (Wang et al., 2024): time-causal residual mask $w(t) = \exp(-\varepsilon R(< t))$ with $\varepsilon = 100$. Burgers and BL only.

Table 8. LC-PINN collocation counts and step budgets per PDE family.

PDE	interior	boundary	initial	Adam / L-BFGS
Burgers	1024	200	200	50k / 0
BL (viscous)	1024	200	200	50k / 0
Helmholtz (1D)	1024	100	—	50k / 1.5k
Helmholtz (2D)	4096	400	—	25k / 1.5k
Helmholtz (3D)	8192	600	—	25k / 1.5k
Schrödinger (1D)	1024	64	—	50k / 1.5k

- *PI-DeepONet* (Wang et al., 2021): branch–trunk architecture; branch encodes the wavenumber k via a 4-layer MLP (hidden width 128); trunk encodes the spatial coordinate x via a 4-layer MLP (hidden width 128). Final solution is the inner product of branch and trunk outputs. Loss is the Helmholtz residual on the same collocation grid as the per- k LC-PINN, summed over an n_k -grid of branch inputs ($n_k = 20$). Adam phase 50k iterations followed by L-BFGS phase 1.5k iterations with a revert-on-worse safety check; on 1D Helmholtz one seed (seed 1) diverged in L-BFGS and was reverted to the Adam-phase weights, reported as such in the 5-seed mean of Table 1.
- *FNO* (Li et al., 2021): width 64, 64 Fourier modes, 4 SpectralConv1d + Conv1d-shortcut blocks (~ 1.07 M parameters). Optimiser: Adam, $\eta = 10^{-3}$, weight decay 10^{-4} , StepLR with step = epochs/5 and $\gamma = 0.5$. Loss: per-sample rel- L^2 . Training set: 512 random Fourier initial conditions (truncated 6-mode spectrum, decaying amplitude) solved by the same Radau scheme used for the reference; 2000 epochs.

Reference solutions. Burgers reference: Fourier-spectral spatial discretisation with a Radau IIA implicit time integrator, integrated to relative residual $< 10^{-6}$ on a 512×1001 space–time grid. Buckley–Leverett reference: explicit Lax–Friedrichs flux on $n_x = 1000$ with centred-difference viscous regularisation, CFL-limited time stepping. Helmholtz 1D reference: closed-form manufactured solution $u(x; k) = \sin(\pi x) \cos(kx)$ with the forcing f chosen to match. Helmholtz 2D reference: closed-form manufactured solution $u(x, y; k) = \sin(\pi x) \sin(\pi y) \cos(kx) \cos(ky)$. Letting $a(x; k) = \sin(\pi x) \cos(kx)$ and $b(y; k) = \sin(\pi y) \cos(ky)$, so $u = a(x; k) b(y; k)$, the forcing

$$f(x, y; k) = \Delta u + k^2 u = -(2\pi^2 + k^2) u - 2\pi k [\cos(\pi x) \sin(kx) \sin(\pi y) \cos(ky) + \sin(\pi x) \cos(kx) \cos(\pi y) \sin(ky)]$$

agrees with a centred-difference numerical Laplacian on the manufactured solution to absolute error $< 4 \times 10^{-6}$. Helmholtz 3D reference: closed-form manufactured solution $u(x, y, z; k) = \sin(\pi x) \sin(\pi y) \sin(\pi z) \cos(kx) \cos(ky) \cos(kz)$ on the unit cube, with the matching forcing $f = \Delta u + k^2 u$ derived analytically; agreement to a centred-difference numerical Laplacian is $< 10^{-5}$ across $k \in \{1, 3, 5\}$. Schrödinger reference: with $u(x; \alpha) = \sin(\pi x) g(x; \alpha)$, $g(x; \alpha) = e^{-\alpha(x-1/2)^2/2}$, the forcing is

$$f(x; \alpha) = -u'' + V(x; \alpha) u = (\pi^2 + \alpha) u + 2\pi\alpha(x - \frac{1}{2}) \cos(\pi x) g(x; \alpha),$$

which agrees with a centred-difference numerical residual to $< 5 \times 10^{-6}$ across $\alpha \in \{0.5, 2, 5, 10\}$.

Evaluation grids. Burgers / BL test errors are computed on a 256×101 space–time grid; Helmholtz on a 1024-point spatial grid. The K -shot averaged LC-PINN prediction is the mean of K forward passes at independent λ samples from p_λ ; we use $K = 100$ for Burgers and $K = 25$ for BL in the headline tables.

Compute. Each run uses a single Apple M4 Max with the PyTorch MPS backend. Reported wall times are end-to-end per seed (model construction, training, and evaluation), measured with `time.perf_counter`.

Reproducibility. Random seeds: $\{0, 1, 2, 3\}$ for the four headline replicates; $\{0, 1\}$ for the 2-seed ablation budgets. Each seed controls the PyTorch global RNG and the NumPy RNG used to draw collocation points and λ -samples. The full training script, equation modules, and reference solvers are released with the camera-ready version.

C. Ablations: full protocol and numbers

K -shot averaging. Sweeping $K \in \{25, 50, 100, 200, 400\}$ on the trained LC-PINN gives essentially constant rel- L^2 (3.51×10^{-3} at $K = 25$, 3.49×10^{-3} at $K = 100$, 3.49×10^{-3} at $K = 400$; spread $\pm 2.7 \times 10^{-3}$ stays roughly constant). At the optimum each λ -slice is already a residual minimiser (Proposition 1), so increasing K adds samples but no new information.

Table 9. Per- α rel- L^2 on 1D parametric Schrödinger (mean $\pm$ std over 4 LC seeds and 2 SA seeds per α). Bold: best per row.

α	SA-PINN (per- α)	LC-PINN (one net)
0.5	$(5.89 \pm 1.26) \times 10^{-5}$	$(3.69 \pm 3.23) \times 10^{-4}$
5.0	$(1.07 \pm 1.05) \times 10^{-2}$	$(7.50 \pm 4.25) \times 10^{-5}$
10.0	$(9.29 \pm 6.67) \times 10^{-4}$	$(1.54 \pm 0.72) \times 10^{-4}$
grid mean	3.89×10^{-3}	1.99×10^{-4}

n_λ per training step. At a matched 25k-epoch budget on Burgers (2 seeds), sweeping $n_\lambda \in \{1, 4, 16\}$ per step gives non-monotone behaviour: $n_\lambda = 1 \rightarrow 1.69 \times 10^{-2}$, $n_\lambda = 4 \rightarrow 1.22 \times 10^{-2}$, $n_\lambda = 16 \rightarrow 1.50 \times 10^{-1}$. Too few λ per step makes the gradient estimate of the p_λ -expectation noisy (Remark 3 / Hoeffding); too many concentrates compute into fewer optimisation steps and the network does not converge in the fixed budget. At the headline 50k-epoch budget we use $n_\lambda = 4$.

Sampling distribution p_λ . At matched 25k-epoch / $n_\lambda = 4$ / 2-seed budget on Burgers, swapping p_λ from uniform on the simplex to log-uniform gives rel- L^2 of $1.22 \times 10^{-2} \pm 5.14 \times 10^{-3}$ (uniform) versus $1.74 \times 10^{-1} \pm 9.26 \times 10^{-3}$ (log-uniform), a $\sim 14\times$ degradation. We read this not as a contradiction with Remark 2 (an asymptotic-optimum statement) but as a finite-budget effect: log-uniform places dominant mass on small loss weights, weakening the gradient signal at large λ values where the residual contributes most to the test-relevant slices.

D. FiLM does not help in loss-weight mode

In the parametric-coefficient mode (Helmholtz, Schrödinger) FiLM modulation lifts the LC-PINN by an order of magnitude: on 1D Helmholtz, plain concat reaches rel- $L^2 \sim 9 \times 10^{-3}$ on the same k -grid (lc_pinn_helmholtz runs), whereas FiLM+L-BFGS reaches 9.4×10^{-4} (Table 1, $\sim 10\times$ tighter). In the loss-weight mode (Burgers) it regresses: a four-seed re-run of Burgers with FiLM+L-BFGS gave $5.7 \times 10^{-2} \pm 5.5 \times 10^{-2}$, with two seeds at the concat baseline ($\sim 3 \times 10^{-3}$) and two trapped near 10^{-1} . We read this as evidence that FiLM’s value lies in disentangling *operator*-changing inputs from spatial coordinates; loss-weight inputs do not change the operator, so FiLM gates over a degree of freedom that does not exist. The headline Burgers and BL rows in Section 5.5 therefore use the simpler concat+Adam configuration of Dosovitskiy & Djolonga (2020).

E. Per-parameter breakdowns

The per- α breakdown for 1D Schrödinger (Section 5.2) is given in Table 9 and visualised in Figure 3. The two amortised methods (LC and DON) hold a flat $\sim 10^{-4}$ curve across the entire $\alpha \in [0.5, 10]$ range; the per- α SA-PINN baseline matches at $\alpha = 0.5$ and $\alpha = 10$ but collapses by two orders of magnitude at the intermediate $\alpha = 5.0$ training point — the same incoherence pattern documented for Helmholtz in Section 5.1.

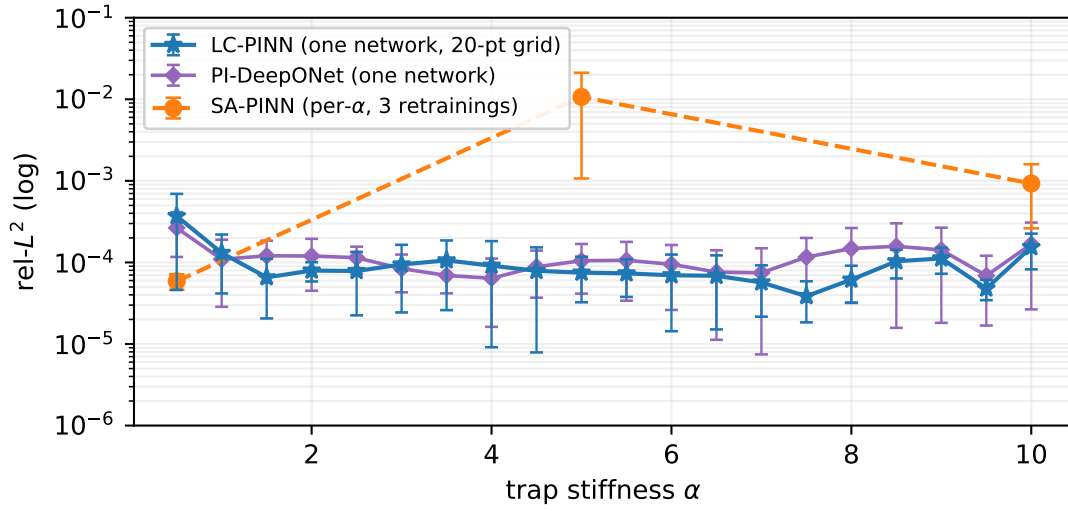


Figure 3. Per- α $\text{rel-}L^2$ on 1D parametric Schrödinger. LC-PINN (blue, 4 seeds) and PI-DeepONet (purple, 4 seeds) cover the 20-point grid from one trained network each; SA-PINN (orange, 2 seeds) is three separate retrainings at $\alpha \in \{0.5, 5.0, 10.0\}$. Error bars are $\pm \text{std}$ on a log axis.